

Cell 1 – Load dependencies

```
import random
import numpy as np
import tensorflow as tf
import keras

from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout # new!
from tensorflow.keras.layers import BatchNormalization # new!

from keras.optimizers import Adam
from matplotlib import pyplot as plt
import pandas as pd

seed = 42
random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)
```

Cell 2 – Load data

```
(x_train, y_train), (x_valid, y_valid) = mnist.load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 0s 0us/step

Cell 3 – Preprocess data

```
x_train = x_train.reshape(60000, 784).astype('float32')
x_valid = x_valid.reshape(10000, 784).astype('float32')

x_train /= 255
x_valid /= 255

n_classes = 10
y_train = keras.utils.to_categorical(y_train, n_classes)
y_valid = keras.utils.to_categorical(y_valid, n_classes)
```

Cell 4 – CA hyperparameters (use the "best" combo here)

```
N_HIDDEN_LAYERS = 3
HIDDEN_UNITS = 64
ACTIVATION = 'relu'
LEARNING_RATE = 0.001
```

```
EPOCHS = 20
```

```
BATCH_SIZE = 128
```

```
def build_model(dropout_rate):
    model = Sequential()

    model.add(Dense(HIDDEN_UNITS, activation=ACTIVATION, input_shape=(784,)))
    model.add(BatchNormalization())

    for _ in range(N_HIDDEN_LAYERS - 1):
        model.add(Dense(HIDDEN_UNITS, activation=ACTIVATION))
        model.add(BatchNormalization())

    model.add(Dropout(dropout_rate))
    model.add(Dense(n_classes, activation='softmax'))

    return model
```

```
# Cell 5 – Run experiments for dropout (.1, .2, .3, .4, .5) and collect accuracy
```

```
dropout_rates = [0.1, 0.2, 0.3, 0.4, 0.5]
```

```
results = []
```

```
histories = {}
```

```
for dr in dropout_rates:
    print("\n" + "=" * 60)
    print(f"Training with dropout = {dr:.1f}")
    print("=" * 60)

    model = build_model(dr)

    optimizer = Adam(learning_rate=LEARNING_RATE)
    model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])

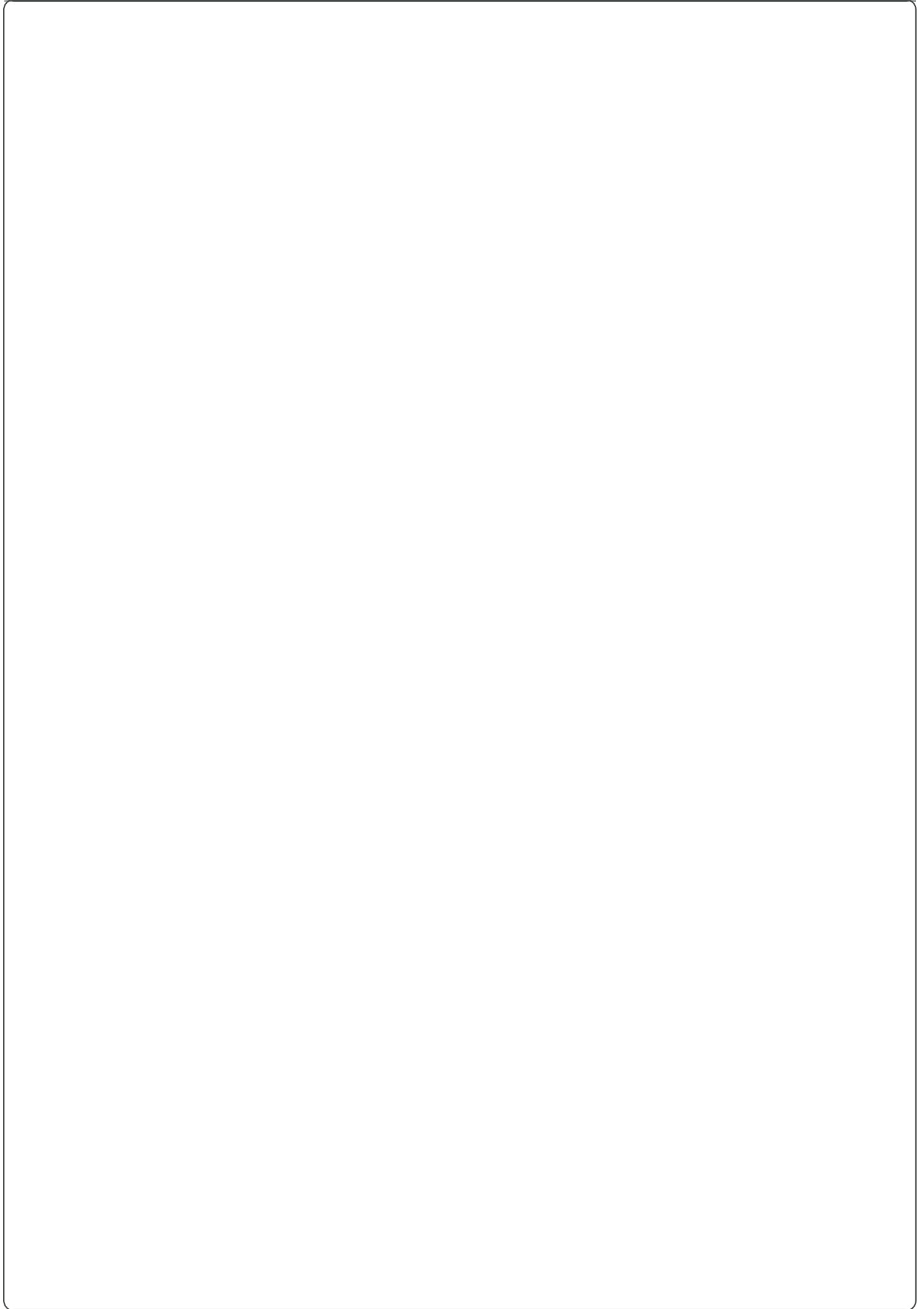
    history = model.fit(
        x_train, y_train,
        batch_size=BATCH_SIZE,
        epochs=EPOCHS,
        verbose=1,
        validation_data=(x_valid, y_valid)
    )

    val_loss, val_acc = model.evaluate(x_valid, y_valid, verbose=0)

    results.append({
        "#Hidden layer": N_HIDDEN_LAYERS,
        "Activation Function": ACTIVATION,
        "Learning Rate": LEARNING_RATE,
        "Dropout": dr,
        "Accuracy": val_acc
    })

    histories[dr] = history
```

```
df_results = pd.DataFrame(results)
df_results["Accuracy"] = (df_results["Accuracy"] * 100).round(2)
df_results
```



```
=====
Training with dropout = 0.1
=====
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/20
469/469 ██████████ 13s 13ms/step - accuracy: 0.7964 - loss: 0.6672 - val_accuracy:
Epoch 2/20
469/469 ██████████ 4s 8ms/step - accuracy: 0.9556 - loss: 0.1508 - val_accuracy:
Epoch 3/20
469/469 ██████████ 3s 6ms/step - accuracy: 0.9677 - loss: 0.1044 - val_accuracy:
Epoch 4/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9758 - loss: 0.0769 - val_accuracy:
Epoch 5/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9818 - loss: 0.0591 - val_accuracy:
Epoch 6/20
469/469 ██████████ 3s 5ms/step - accuracy: 0.9856 - loss: 0.0467 - val_accuracy:
Epoch 7/20
469/469 ██████████ 3s 7ms/step - accuracy: 0.9871 - loss: 0.0399 - val_accuracy:
Epoch 8/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9897 - loss: 0.0343 - val_accuracy:
Epoch 9/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9896 - loss: 0.0316 - val_accuracy:
Epoch 10/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9905 - loss: 0.0285 - val_accuracy:
Epoch 11/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9922 - loss: 0.0233 - val_accuracy:
Epoch 12/20
469/469 ██████████ 3s 6ms/step - accuracy: 0.9919 - loss: 0.0238 - val_accuracy:
Epoch 13/20
469/469 ██████████ 3s 6ms/step - accuracy: 0.9930 - loss: 0.0205 - val_accuracy:
Epoch 14/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9935 - loss: 0.0190 - val_accuracy:
Epoch 15/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9935 - loss: 0.0181 - val_accuracy:
Epoch 16/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9948 - loss: 0.0152 - val_accuracy:
Epoch 17/20
469/469 ██████████ 3s 6ms/step - accuracy: 0.9954 - loss: 0.0137 - val_accuracy:
Epoch 18/20
469/469 ██████████ 3s 6ms/step - accuracy: 0.9952 - loss: 0.0149 - val_accuracy:
Epoch 19/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9961 - loss: 0.0114 - val_accuracy:
Epoch 20/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9962 - loss: 0.0111 - val_accuracy:
=====
```

```
Training with dropout = 0.2
=====
Epoch 1/20
469/469 ██████████ 6s 8ms/step - accuracy: 0.7776 - loss: 0.7381 - val_accuracy:
Epoch 2/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9521 - loss: 0.1624 - val_accuracy:
Epoch 3/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9654 - loss: 0.1149 - val_accuracy:
Epoch 4/20
469/469 ██████████ 3s 5ms/step - accuracy: 0.9733 - loss: 0.0899 - val_accuracy:
Epoch 5/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9789 - loss: 0.0696 - val_accuracy:
Epoch 6/20
```

Epoch 6/20
469/469 ————— 3s 7ms/step - accuracy: 0.9827 - loss: 0.0573 - val_accuracy:
Epoch 7/20
469/469 ————— 2s 5ms/step - accuracy: 0.9838 - loss: 0.0509 - val_accuracy:
Epoch 8/20
469/469 ————— 2s 5ms/step - accuracy: 0.9864 - loss: 0.0433 - val_accuracy:
Epoch 9/20
469/469 ————— 3s 5ms/step - accuracy: 0.9886 - loss: 0.0359 - val_accuracy:
Epoch 10/20
469/469 ————— 2s 5ms/step - accuracy: 0.9904 - loss: 0.0299 - val_accuracy:
Epoch 11/20
469/469 ————— 4s 7ms/step - accuracy: 0.9894 - loss: 0.0317 - val_accuracy:
Epoch 12/20
469/469 ————— 2s 5ms/step - accuracy: 0.9918 - loss: 0.0260 - val_accuracy:
Epoch 13/20
469/469 ————— 2s 5ms/step - accuracy: 0.9912 - loss: 0.0259 - val_accuracy:
Epoch 14/20
469/469 ————— 2s 5ms/step - accuracy: 0.9927 - loss: 0.0218 - val_accuracy:
Epoch 15/20
469/469 ————— 2s 5ms/step - accuracy: 0.9923 - loss: 0.0218 - val_accuracy:
Epoch 16/20
469/469 ————— 3s 7ms/step - accuracy: 0.9946 - loss: 0.0167 - val_accuracy:
Epoch 17/20
469/469 ————— 4s 5ms/step - accuracy: 0.9929 - loss: 0.0209 - val_accuracy:
Epoch 18/20
469/469 ————— 3s 5ms/step - accuracy: 0.9939 - loss: 0.0170 - val_accuracy:
Epoch 19/20
469/469 ————— 2s 5ms/step - accuracy: 0.9950 - loss: 0.0137 - val_accuracy:
Epoch 20/20
469/469 ————— 4s 8ms/step - accuracy: 0.9939 - loss: 0.0174 - val_accuracy:

=====
Training with dropout = 0.3
=====

Epoch 1/20
469/469 ————— 5s 6ms/step - accuracy: 0.7560 - loss: 0.8031 - val_accuracy:
Epoch 2/20
469/469 ————— 2s 5ms/step - accuracy: 0.9476 - loss: 0.1784 - val_accuracy:
Epoch 3/20
469/469 ————— 3s 6ms/step - accuracy: 0.9629 - loss: 0.1251 - val_accuracy:
Epoch 4/20
469/469 ————— 3s 6ms/step - accuracy: 0.9696 - loss: 0.1009 - val_accuracy:
Epoch 5/20
469/469 ————— 2s 5ms/step - accuracy: 0.9751 - loss: 0.0815 - val_accuracy:
Epoch 6/20
469/469 ————— 3s 5ms/step - accuracy: 0.9803 - loss: 0.0664 - val_accuracy:
Epoch 7/20
469/469 ————— 2s 5ms/step - accuracy: 0.9811 - loss: 0.0594 - val_accuracy:
Epoch 8/20
469/469 ————— 3s 6ms/step - accuracy: 0.9842 - loss: 0.0505 - val_accuracy:
Epoch 9/20
469/469 ————— 3s 5ms/step - accuracy: 0.9863 - loss: 0.0441 - val_accuracy:
Epoch 10/20
469/469 ————— 2s 5ms/step - accuracy: 0.9874 - loss: 0.0392 - val_accuracy:
Epoch 11/20
469/469 ————— 2s 5ms/step - accuracy: 0.9892 - loss: 0.0329 - val_accuracy:
Epoch 12/20
469/469 ————— 2s 5ms/step - accuracy: 0.9904 - loss: 0.0299 - val_accuracy:
Epoch 13/20
469/469 ————— 3s 7ms/step - accuracy: 0.9903 - loss: 0.0298 - val_accuracy:
Epoch 14/20

```
469/469 ██████████ 2s 5ms/step - accuracy: 0.9923 - loss: 0.0243 - val_accuracy:
Epoch 15/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9930 - loss: 0.0210 - val_accuracy:
Epoch 16/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9925 - loss: 0.0213 - val_accuracy:
Epoch 17/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9930 - loss: 0.0210 - val_accuracy:
Epoch 18/20
469/469 ██████████ 3s 7ms/step - accuracy: 0.9930 - loss: 0.0196 - val_accuracy:
Epoch 19/20
469/469 ██████████ 4s 5ms/step - accuracy: 0.9940 - loss: 0.0172 - val_accuracy:
Epoch 20/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9947 - loss: 0.0152 - val_accuracy:
```

```
=====
Training with dropout = 0.4
=====
```

```
Epoch 1/20
469/469 ██████████ 6s 8ms/step - accuracy: 0.7387 - loss: 0.8625 - val_accuracy:
Epoch 2/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9449 - loss: 0.1903 - val_accuracy:
Epoch 3/20
469/469 ██████████ 3s 5ms/step - accuracy: 0.9611 - loss: 0.1357 - val_accuracy:
Epoch 4/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9683 - loss: 0.1063 - val_accuracy:
Epoch 5/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9745 - loss: 0.0871 - val_accuracy:
Epoch 6/20
469/469 ██████████ 3s 7ms/step - accuracy: 0.9777 - loss: 0.0746 - val_accuracy:
Epoch 7/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9811 - loss: 0.0632 - val_accuracy:
Epoch 8/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9825 - loss: 0.0580 - val_accuracy:
Epoch 9/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9858 - loss: 0.0492 - val_accuracy:
Epoch 10/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9873 - loss: 0.0432 - val_accuracy:
Epoch 11/20
469/469 ██████████ 3s 7ms/step - accuracy: 0.9875 - loss: 0.0398 - val_accuracy:
Epoch 12/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9884 - loss: 0.0346 - val_accuracy:
Epoch 13/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9896 - loss: 0.0332 - val_accuracy:
Epoch 14/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9908 - loss: 0.0271 - val_accuracy:
Epoch 15/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9918 - loss: 0.0265 - val_accuracy:
Epoch 16/20
469/469 ██████████ 4s 7ms/step - accuracy: 0.9907 - loss: 0.0278 - val_accuracy:
Epoch 17/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9920 - loss: 0.0236 - val_accuracy:
Epoch 18/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9930 - loss: 0.0202 - val_accuracy:
Epoch 19/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9937 - loss: 0.0178 - val_accuracy:
Epoch 20/20
469/469 ██████████ 2s 5ms/step - accuracy: 0.9945 - loss: 0.0168 - val_accuracy:
```

```
=====
Training with dropout = 0.5
=====
```

Cell 6 – Pick best dropout + draw the model history (same plotting style as slides)

```
best_row = df_results.sort_values("Accuracy", ascending=False).iloc[0]
best_dropout = float(best_row["Dropout"])
print("Best dropout:", best_dropout, "| Accuracy:", best_row["Accuracy"])
```

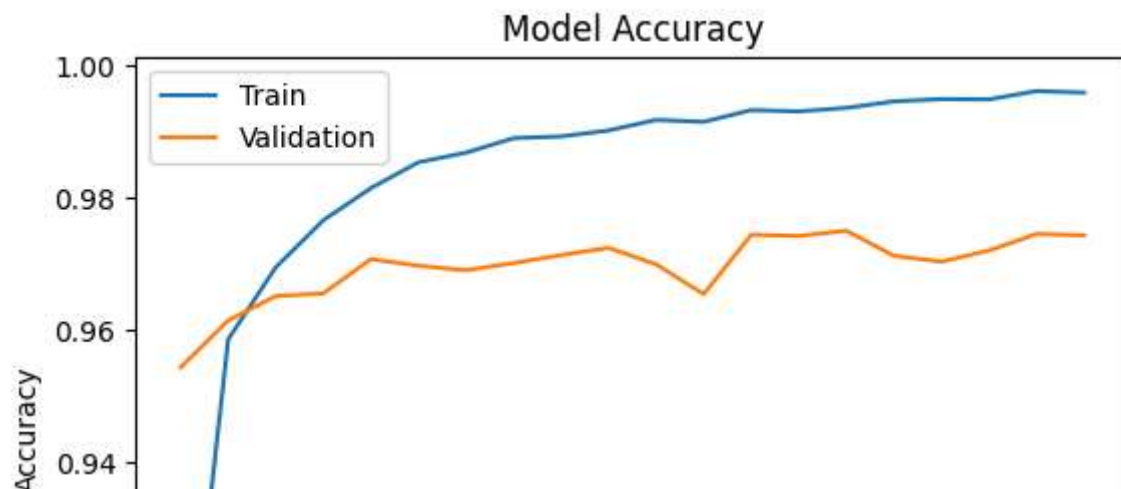
```
history = histories[best_dropout]
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss/Error')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

```
# print(history.history.keys())
# pd.DataFrame(history.history).plot(figsize=(8,5))
# plt.show()
```

Epoch 1/20 Best dropout: 0.1 | Accuracy: 97.43



- val_accuracy:
- val_accuracy:
- val_accuracy:
- val_accuracy:

